SHARED TECHNICAL, MONETIZATION AND LAUNCH SPECIFICATION

Version 1.0 | 20 Oct 2026 | Applies to CASHFLOW CHAINS and BLACK BAG

> A shipping technical specification, not a guarantee of revenue or store acceptance. Game-specific rules are defined in each game's master specification. When the two differ, game-specific rules take precedence unless this document concerns safety, payment security, or platform compliance.

S01. Product goals and non-goals

Goals: polished original 2D mobile games with satisfying sub-three-minute rounds; meaningful 15-minute play and optional hours-long skill challenges; persistent collections; entirely playable in single-player; varied offers with demonstrable value; measurable purchasing funnels and stable offline saves; releases that can be maintained by one operator with Codex.

Non-goals for v1: multiplayer, account-gated first launch, user-generated publishing, open-world simulation, deep NPC AI, realistic 3D, metaverse systems, subscription-only access, paid randomized loot crates, real-money gambling, daily punishment, loss of owned permanent goods while offline, aggressive notification loops or hidden variable pricing.

Success is an empirical business outcome. We do not invent a forecast or promise a revenue floor. Validation is sequential: works -> fun -> return -> spend -> economical to acquire.

S02. Engine versions and device baseline

- Pin Unity **6.3 LTS** or a verified newer supported 6.x patch; commit `ProjectVersion.txt`, `packages-lock.json` and editor version instructions. Use the **Universal 2D** template with URP 2D Renderer; rendering camera is orthographic and Z coordinates used only for draw order.
- Compile `IL2CPP`, ARM64, Android AAB and release keystore handling according to Play Console. New Google Play submissions from Aug 31 2026 target **API 36+**. Verify the installed Unity editor, Android SDK and plugins support this target. Set min API based on actual testing; proposed API 26+ (not mandated).
- iOS: maintain compatible project and automated builds where Mac/Xcode/signing available; target iOS version based on supported Unity/editor and testing. Android-first beta is a business sequencing choice, not a permanent restriction.
- Reference virtual portrait design: 1080x1920 at 9:16, test taller and shorter devices 720x1600, 1080x2340, 1170x2532, tablets and notches. Safe-area padding on every gameplay/control screen. No essential information behind status bars or home indicators.
- Performance: target sustained 60 fps on mainstream test phone, acceptable stable 30 fps on modest devices, main thread budget ~16.7 ms at 60 fps, avoid allocations in per-frame loops. Measure rather than claim. Battery, thermal, app resumes, audio interruptions are release gates.
- Accessibility: readable min 14sp-equivalent text where practical; touch targets about 44–48 logical px or larger; reduced motion option; color-blind safe cues; no color-only state; high contrast; sound/

vibration toggles; screen reader labels on native UI where feasible; one-handed gestures with tap alternatives.

S03. Platform boundary and plugin assumptions

Official Unity Codex plugin is a **skills/instruction integration**, not a magic asset synthesizer; Codex must run Unity CLI/editor commands and inspect results. Unity's 2D and Sprite Atlas packages can make 2D worlds efficiently. Lock Unity and package versions and inspect compile warnings. Do not paste obsolete Unity IAP 4 `IStoreListener` patterns into a Unity IAP 5 project. Use version-specific Unity IAP `StoreController` lifecycle.

External SDKs must be listed with versions, license, data collection and platform support in `Docs/THIRD_PARTY.md`. Third-party downloaded graphics, paid packs or copied proprietary icons are forbidden. Original art can be made via SVG paths, code-drawn raster/vector elements, original image generation with rights checks, procedural particles and authored gradient/texture systems. Image-generated composition is not necessarily a workable sprite sheet: require transparent cutouts and final in-engine review.

S04. Project architecture

One repository per game:

```
Assets/_Game/{Art,Audio,Data,Editor,Fonts,Materials,Prefabs,Scenes,Scripts,Tests,UI}
Assets/_Game/Scripts/{Domain,Application,Presentation,Infrastructure,Monetization,Analytics}
Docs/{Design,ArtBible,Economy,Analytics,ContentCatalog,Release,Incidents}
Tools/{ContentValidation,SaveInspection,SimulationHarness,AssetGeneration}
Builds/                 # ignored, not committed
ProjectSettings/
Packages/
AGENTS.md
```

`Domain` = pure, Unity-independent C# model with deterministic RNG, state transition reducers, game rules and invariant checks. `Application` = commands, events, persistence coordination, progression/offer eligibility. `Presentation` = UI/renderer/input/animations without game economy authority. `Infrastructure` = local storage, cloud adapters, telemetry and clock. `Monetization` = native billing, catalog, receipts, entitlements. Data defined as ScriptableObjects generated from canonical JSON definitions, validated in editor and CI. Domain never imports UnityEngine.

Required interfaces (conceptual, actual API may adapt)

- `IGameRuleEngine.Step(GameState state, PlayerCommand command) -> StepResult` with immutable or carefully owned state mutation and domain events.
- `IClock.UtcNow()` and monotonic session timer for gameplay; mockable in tests.
- `IRandomSource.NextInt(seed, bounds)` seeded deterministic generator for challenge replay and save migration.
- `IStateStore.SaveAtomically(slot, payload)` and load with schema version/hash/backups.
- `ICommerceProvider` mock, Android and iOS adapters, transaction status observers.
- `IAnalyticsSink.Enqueue(typedEvent)` plus consent policy and offline queue.
- `IOfferPolicy` checks player progression, eligibility, frequency caps and platform price; can be disabled remotely.
- `IContentCatalog` for versioned rules, missions, progression and product data.

UI must never change wallet values without domain command validation. Each reward is traceable to a unique operation/reward transaction ID. Preserve events for debugging and customer support without storing sensitive personal information.

S05. Save, offline progression and migrations

- Single-user anonymous local save created on first interaction. Save after each round and important progression/purchase grant, on app suspend, and at periodic safe checkpoints. Use `tmp` write + fsync or platform-equivalent + atomic rename; retain last known good backup.
- Save includes `schemaVersion`, `contentVersion`, `saveId`, `playerIdLocal`, currency ledgers, owned IDs, collected goals, current play state, replay seed, last trusted UTC timestamp, completed reward IDs, and completed purchase transaction IDs (or hashed receipt refs). No card numbers.
- Load validation checks bounded arrays, NaN/infinity, duplicate IDs, current content compatibility, count conservation, and maximum legitimate amounts; recover from backup on corruption with clear UX.
- **Offline progress is limited and transparent**. No infinite passive money, no irreversible loss. Catch-up computed from capped elapsed time, capacity and completed orders, not `earningsRate * rawWallClock` blindly; reject negative time and clamp impossible clock jumps. Do not depend on device clock for high-value competitive events.
- First launch without cloud login. Optional cloud backup later, with account deletion/export paths if collected data requires it. Purchases of permanent entitlements validated against store and restored after reinstall; consumable cross-device support needs server-backed ledger or explicit locally-limited warning and support policy.

S06. Art and identity non-negotiables

Pure flat 2D, no perspective transforms, no isometric render, no 3D mesh, no fake rounded 3D toys. Concept screenshots are inspiration for color and polish, **not literal build specifications**. Prior concept shots sometimes contained angled roofs, impossible inventory geometry or decorative UI: correct these in actual game.

Cashflow: vivid midnight teal/indigo, mint/neon-chartreuse money-flow lines, warm amber economic reactions; 2D orthographic top-down **flat building symbols** whose shape and iconography are clear. Decorative city skyline is a flat distant illustration, not the board camera. Visual distinctiveness comes from kinetic economic circuitry, miniature typography, radial bursts and precise animated node paths. Avoid imitating identifiable property/character designs from other games.

Black Bag: luxurious editorial paper-cut/engraved **flat 2D** style: crimson-black leather palette, antique brass line work, cream parchment, jewel blue glows and hand-inked object silhouettes. Packing area is a flat grid on top of a drawn suitcase; valuables are flat silhouettes with ornamental interior patterns, not shaded 3D photo objects. Atmospheric museum silhouette can be a static vector illustration behind the HUD. Atmosphere is through ink patterns, candle-flicker overlaid sprites, page reveals and subtle particles.

Programmatic asset quality gates

1. Stable art bible defines palette hex, stroke bands, padding, shadow, ortho/perspective prohibition, object readability at 48–96px, export standards.
2. Generate a **small representative vertical-slice set** first: Cashflow 8 flat buildings + 4 icons + full board; Black Bag 18 polyomino item silhouettes + 3 panels + full packing grid.

3. Export sprites with correct transparent alpha, sRGB/color management, consistent pixels-per-unit/9-slice metadata, pivot positions and asset IDs; no text baked into gameplay sprites unless localization independent.
4. Build sprite atlases, check runtime atlas packing, overdraw, compression artifacts, touch visibility, legibility against animated backgrounds. Avoid mipmap/filter bleed for small UI sprites.
5. Generate actual screenshot captures from Unity across target devices; compare against style gate. Reject inconsistent sprite scales, off-perspective perspective, unreadable glow and excessive effects.
6. Produce original audio via procedural synthesis, licensed self-generated audio or model-generated audio for which permissions are documented. Cache assets before ship; no runtime inference dependency.

S07. Data-driven content system

Use JSON schemas validated by tests; import into game data ScriptableObjects. Stable string IDs and localized text keys; never derive item identity from visible names. Content rows include `id`, `versionAdded`, `tags`, `unlockRule`, `artKey`, `sfxKey`, `rarity` if applicable, `testTags`, and `balanceTier`. Mark deprecated content without deleting owned items. No self-contradictory unlock graph, unknown reference IDs or impossible achievements.

Content pipeline: content authoring -> JSON schema validation -> simulator property tests -> Unity import -> headless build -> play-mode screenshot tests -> human review -> release version. Use `Tools/ContentValidation/` to run 10k generated rounds/vaults in batch and report exploit or unwinnable patterns.

S08. Monetization objectives and ethical constraints

Main business metrics: **net revenue per acquired install**, D1/D7/D30 retention, payer rate by cohort, ARPPU, repeat purchase rate, ads ARPDAU if used, paywall/offer exposure to conversion, refunds, opt-outs and churn. No claim any price/timing is optimal before A/B experiments.

All products have clear player benefit. At minimum, one small starter product, one permanent-value product, one repeatable convenience product, one aesthetic/collection item or gameplay content pack. Larger expansion/passes require enough worthwhile content. Avoid purchases that entirely bypass the main fun, force ads, selling immunity from artificial punishment, fake discount comparisons, pay-to-remove intentionally broken systems, social shaming, personalized exploitative offers, and randomized paid packages at launch. Provide plain direct-currency purchase options rather than opaque exchange ladders.

Prices must come from Google Play/App Store product metadata, not hardcoded USD in purchase buttons. Suggested tiers are **hypotheses** in game-specific files, localized via store config. The pricing and balance are subject to legal review and live testing.

Shared product life cycle

`unavailable -> fetch_products -> ready -> clicked -> platform_sheet -> pending/deferred/failed -> verify -> idempotent_grant -> confirm -> owned/available`.

- Disable paid confirmation when no store connectivity; do not block gameplay.
- Use Unity IAP v5 event-driven `StoreController` model, version-specific APIs, `OnPurchasePending`, purchase fetch and confirmation behavior. Never confirm unfulfilled consumable orders. Handle crash redelivery without duplicate items.

- For consumables use server-backed account/transaction ledger if offering durable cross-device support; if v1 local-only, make limitation explicit and do not claim reliable cross-device restore of spent consumables. Non-consumables must restore through store entitlements.
- Reconcile refunds/revocations, deferred family approvals, cancellation and transaction retries; user-facing support and no duplicate grants. No purchase UI until pricing metadata loaded.
- Purchase receipts / verification secrets belong in backend secure secrets manager, not client source, build repo or remote config.
- Region-dependent billing and disclosures must be reviewed; Google Play Billing for digital goods unless eligible approved alternative program, Apple IAP for digital unlocks as applicable.

S09. Offer eligibility and on-screen frequency

- No offer until the player completes an enjoyable first run and voluntarily opens a related progression feature. Starter pack can become eligible after 10–20 mins or equivalent progression, but never forced.
- Pop-up promotion maximum once per player session, no pop-ups during an active round, purchase after failure not automatically suggested; store icons accessible but nonintrusive. User dismissal honored for at least several sessions; no genuine purchase value should depend on false timers.
- Offers have `eligibilityMilestones`, `minSessionIndex`, `minDaysSinceInstall`, `cooldownSessions`, `maxImpressions`, `maxPurchases`, `validContentVersion` and `availableRegions`/store approval. Remote config may choose compliant eligible variants; remote config must not secretly replace purchased entitlements or alter core value without versioning.
- No paid VIP subscription at v1. Later optional event pass only if production pipeline exists and user value is explicit, with fixed visible reward map and no retroactive revocation of purchased permanent items.
- Rewarded ads are optional, initiated by the player, clear benefit and no surprise interstitials. No ad network SDK until privacy/demand testing supports its cost; enforce consent requirements.

S10. Experimental metrics and minimal sample rules

Instrument events: `install_first_open`, `tutorial_step_start`, `tutorial_step_complete`, `round_start`, `round_end`, `round_abandon`, `session_start`, `session_end`, `replay_pressed`, `unlock_earned`, `collection_complete`, `offer_impression`, `offer_dismissed`, `offer_opened`, `purchase_sheet_opened`, `purchase_success`, `purchase_failure`, `entitlement_restored`, `rewarded_ad_offer`, `rewarded_ad_complete`, `refund_received` and game-specific events.

Every event: schema/version, anonymized player/session ID, experiment cell, build/content versions, UTC time, elapsed time, game mode, progression milestone, consent scope. IAP events: SKU, transaction *reference*, local currency code and amount from store reporting (no card data). Never dump PII, secret keys, receipts, or entire device identifiers into analytics.

Test true cohorts, not fake purchase buttons as revenue evidence. Low sample conversion is noisy: report Wilson/binomial CIs, absolute denominators and observation windows. Segment by acquisition source, country, device, content version and purchase status without invasive profiling. A/B each major hypothesis one at a time; use retention and contribution guardrails, not first-hour revenue alone.

Proposed gates (research targets, not promises): tutorial completion ≥85% in controlled new-user test; 3-min completion ≥75%; voluntary next-round desire ≥60% in moderated test; D1 ≥30%, D7 ≥8%, D30 ≥3% as stretch validation; refunds within policy limits and minimal crash rate; target retention should be revised per geography/genre. Prioritize incremental net LTV minus acquisition and support, not raw ARPPU.

S11. Privacy, safety, compliance

- Privacy policy URL, contact/support route, account/data deletion instructions if collecting user accounts or identifiers; Google Data Safety declaration includes **all SDKs** and their collection. Apple App Privacy labels and required privacy manifests; ATT permission *only* if tracking across apps/websites within Apple's definition. Don't invent consent if no tracker is used.
- Fictional heist depiction acceptable subject to store ratings, but avoid real burglary tutorials, realistic criminal instructions, blood/gore or branded real museums/protected cultural artifacts unless licensed. Black Bag's weapons seen in aspirational images are **not** v1 gameplay content. Fictional venues and abstract gadgets only.
- No children-targeted presentation or Kids category by default; choose audience and rating accurately. Avoid child-directed ad targeting; verify current COPPA/GDPR-K and regional requirements with actual practices.
- No automated notification pressure; optional notifications only with consent and clear utility (completed scouting/contract), no fake urgency. User can disable notifications in one step.
- App must expose legal disclosures and pricing, refund routes, credits, privacy notice and support contact. Do not submit guessed declarations to stores.

S12. Store publication checklist

Google Play: verified developer identity, registration, package name, signing/upload key, AAB, API 36+, content rating questionnaire, app access disclosure, privacy policy, Data Safety, screenshots showing actual gameplay, feature graphic/icon, description, age audience, territories, verified product SKUs, taxes/merchant payout, optional ads label, internal and closed tests, production access. New *personal* Play accounts opened after Nov 13 2023 generally require **12 opted-in testers for 14 continuous days** before production-access application. Store approval is not automatic.

Apple: developer membership, App Store Connect app ID/bundle ID, Mac/Xcode build & signing or compatible build provider, age rating, privacy nutrition labels, privacy manifests, App Tracking Transparency if applicable, localized screenshots, description, IAP products submitted/reviewed, TestFlight testing, receipt/restore tests, reviewer demo information, release configuration. Apple's relevant fees/account availability must be checked before committing.

Marketing: truthful short gameplay video captured from actual build; store conversion screenshots from shipping UI; ASO keywords and title trademark search; one concise gameplay promise; no plagiarized or misleading marketing mockups. Test ads with organic and small constrained paid samples; only expand if net contribution/long-term retention justify investment.

S13. Testing tiers

1. **Domain unit tests**: every command, scoring, reward and balance invariant.
2. **Property-based tests**: randomized seeds, 10k+ rounds/cases with independent oracles and conservation rules.

3. **Simulation speed tests**: 10k economy/progression actions in batches; stress saved size and memory.
4. **Unity play-mode**: user gestures, HUD updates, layer ordering, transition, pause/resume, ad/IAP mock states.
5. **Actual device**: touch, notch, 60/30 fps, low-memory callbacks, rotation locked, accessibility, network loss, airplane mode, locale, battery.
6. **Platform sandbox**: deferred IAP, failure, rapid tap, duplicate purchase pending, refunds, restore, force-close, product unavailable, wrong price, locale fallback.
7. **Security and exploit**: integer overflow, arithmetic precision, negative currency, race conditions, object overlap, save corruption, RNG seed manipulation, invalid time, reward replay, tampered product IDs.
8. **Human playtesting**: novice onboarding, voluntary replay, session-length fit, difficulty frustration, net liking of visuals, actual payment behavior.

S14. Build/release acceptance policy

Every milestone has a checklist artifact: commit hash, Unity version, test results, device screenshots, known defects, approved changes, progress video if needed. A feature is *done* only when code and original art render correctly in Unity, tests pass, state persists, and UX is checked on a real mobile device. Codex may mark a task `BLOCKED_EXTERNAL` for signing, developer account, human art review, payments, test cohort, privacy/legal review, or store moderation. Do not falsely mark those complete.

Crash-free sessions target >99.5% during soft launch as a screening threshold; actual reporting depends on sample size. Zero P0 blockers: missing save, currency duplication, non-restorable purchases, endless tutorial, impossible levels, incorrect charges, or platform-policy violations.

S15. Operations after launch

Daily for first two weeks: error/crash review, broken SKU monitoring, IAP acknowledgements, reviews/support, economy exploits, acquisition and performance cohorts; weekly content/balance changes only through tested versioned configs. Maintain rollback release and config kill switches for offers and optional events. Log reasons for refund spikes and drop-offs. Always preserve the ability to play core offline mode if third-party services are down.

Monthly: price/product experiment review, net LTV forecasts by observed cohort (uncertainty stated), content pipeline health, localization QA, SDK upgrade audit and store policy check. Avoid costly content additions without evidence of value.

S16. Primary sources checked 20 October 2026

- Unity Codex plugin (Unity 6+): https://docs.unity.com/en-us/ai/unity-plugin/about-unity-plugin
- Unity 6.3 tech art 2D guidance: https://learn.unity.com/article/2d-in-unity-tech-art-essentials
- Unity Sprite Atlas: https://docs.unity.com/en-us/engine/6000.0/manual/unity2d/sprite/atlas/create-sprite-atlas
- Unity IAP pending/restore: https://docs.unity.com/en-us/iap/purchases
- Google Play Android target API 36: https://support.google.com/googleplay/android-developer/answer/11926878

- Google Play personal-account tests: https://support.google.com/googleplay/android-developer/answer/14151465
- Google Play payments: https://support.google.com/googleplay/android-developer/answer/10281818
- Google Play Data Safety SDK obligations: https://support.google.com/googleplay/android-developer/answer/13323374
- Apple App Review: https://developer.apple.com/app-store/review/guidelines/
- Apple app privacy and ATT: https://developer.apple.com/app-store/user-privacy-and-data-use/
- Google subscription rule: https://support.google.com/googleplay/android-developer/answer/9900533

Re-verify policies before submission; they can change after this version.